

ADR-001 - Selection of the Open-Source LMS Foundation for Syrka

Status: Accepted

Date: 2026-07-07

Decision owners: Architecture / Engineering

Scope: Syrka LMS operational layer and its evolution into AI-native Human Capital Infrastructure

1. Decision

Syrka should use a hybrid approach with Open edX as the primary open-source LMS foundation, not a direct long-lived fork of a traditional LMS.

The recommended foundation is:

1. Adopt Open edX concepts, data/event patterns, XBlock/LTI compatibility, course-authoring semantics, assessment primitives, and selected service boundaries as the Syrka LMS skeleton.
2. Do not make the Open edX monolith the permanent Syrka product core. Instead, wrap it during phase 1 and progressively extract domain capabilities into Syrka-native services on the existing Next.js/Supabase stack.
3. Use LTI 1.3 / Advantage as the integration contract for external tools, assignments, simulations, code workspaces, AI tutors, and assessment engines.
4. Treat the LMS as a capability-data collection engine whose primary product responsibility is to produce normalized learning, evidence, credential, and collaboration events for the Syrka Ontology, Human Capability Graph, Capability Inference Engine, Career Intent Engine, Odyssey, Job Passport, and Workforce Mobility Engine.

This is not a recommendation to become "an Open edX company." It is a recommendation to use Open edX as the strongest available open-source educational operating substrate while protecting Syrka's long-term architecture with an anti-corruption layer, canonical event schema, independent ontology/graph services, and Syrka-native user experiences.

2. Context and constraints

Syrka is not a conventional LMS. The long-term hierarchy is:

Syrka LMS

v

Syrka Ontology

v

Human Capability Graph

v

Capability Inference Engine

v

Career Intent Engine

v

Syrka Odyssey

v

Job Passport

v

Workforce Mobility Engine

v

Cross-Border Capability Corridors

The LMS must therefore provide reliable educational operations while becoming a high-fidelity instrumentation layer. It must capture evidence such as assignment submissions, quiz attempts, rubric outcomes, peer review, forum participation, project artifacts, attendance, collaboration, research outputs, GitHub activity, external credentials, and employer signals.

The target Syrka stack in this repository is already oriented around Next.js application surfaces, API routes, Supabase/PostgreSQL storage, research intelligence, country/institution/employer/student dashboards, skill extraction, adaptive paths, CV/profile generation, fairness checks, and intelligence routes. The selected LMS foundation must therefore be compatible with a future in which the modern Syrka experience lives outside the LMS monolith.

3. Investigation method and limitation

The required repositories were investigated using available repository metadata, public documentation, GitHub-rendered source views, and local repository commands. A direct shallow clone of the six upstream repositories was attempted from the working container, but GitHub clone access failed with CONNECT tunnel failed, response 403. Because this was an infrastructure/network limitation, the architecture decision relies on public repository source/documentation inspection available through GitHub and project documentation rather than local full clones.

Repositories evaluated:

- Moodle - <https://github.com/moodle/moodle.git>
- Canvas LMS - <https://github.com/instructure/canvas-lms.git>
- Open edX - <https://github.com/openedx/openedx-platform.git>
- Chamilo - <https://github.com/chamilo/chamilo-lms.git>
- ILIAS - <https://github.com/ILIAS-eLearning/ILIAS.git>
- Sakai - <https://github.com/sakaiproject/sakai.git>

4. Executive comparison

Platform	Primary stack	License	Architectural shape	Best fit	Critical concern	Syrka score
Moodle	PHP, JS, SQL	GPL-3.0	Plugin-rich modular monolith	Broad institutional LMS and plugin ecosystem	Legacy PHP patterns; modernization friction	7.2
Canvas LMS	Ruby/Rails, JS/React, PostgreSQL	AGPL-3.0	Large Rails monolith with strong APIs	Modern higher-ed LMS operations	AGPL obligations, vendor-shaped complexity	7.7
Open edX	Python/Django, JS/React MFEs, MySQL, services	AGPL-3.0	Modular monolith + services + MFEs + event direction	Scalable online learning platform and extensible education runtime	Heavy operational footprint and complex domain model	8.4
Chamilo	PHP/Symfony/Twig direction, MySQL	GPL-3.0+	Simpler PHP LMS	Lightweight deployments	Smaller ecosystem and weaker AI/event foundation	5.6
ILIAS	PHP, MySQL/MariaDB	GPL-3.0	Mature modular monolith	European institutional LMS with governance	Modernization and developer ramp-up	6.2
Sakai	Java, Maven, Tomcat, RDBMS	ECL-2.0	Service-oriented Java tool framework	Higher-ed collaboration and extensibility	Older Java web architecture and smaller modern frontend story	6.6

5. Candidate analyses

5.1 Moodle

Overview. Moodle is the most widely recognized open-source LMS. It targets schools, universities, governments, NGOs, and enterprises. It is GPL-3.0, written primarily in PHP with JavaScript, uses relational databases, and supports self-hosted deployment. It has a large plugin ecosystem, long release history, and broad global community.

Architecture. Moodle is a plugin-based modular monolith. Core subsystems provide course, enrolment, authentication, role/capability, gradebook, files, messaging, events, logging, and web services. Plugins extend activities, blocks, reports, enrolment, auth, repositories, themes, local customizations, and admin tools.

Browser / mobile / integrations

v

Moodle PHP application

v

Core subsystems: course, user, role, gradebook, files, events, web services

v

Plugin types: mod, block, local, report, enrol, auth, repository, theme

v

RDBMS + file storage

Repository and implementation shape. Moodle has a broad root-level layout organized around product subsystems and plugin types. The mod family implements activity modules; auth, enrol, grade, course, user, files, message, calendar, report, admin, and lib carry the core.

Database installation and upgrade code is distributed through component db directories. Tests include PHPUnit and Behat. Configuration is PHP-centric.

Core LMS coverage. Moodle is very strong in authentication, user/role management, enrolment, course management, assignments, quizzes, gradebook, messaging, notifications, calendar, file storage, permissions, reports, and learning analytics. Its role/capability system is mature.

Extensibility and events. Moodle has a mature Events API, log stores, web services, and many plugin types. Assignment submitted, quiz attempted, forum posted, course viewed, grade updated, and completion events can be observed. It is realistic to stream events into Syrka, but the canonical capability model would need to live outside Moodle.

AI readiness. Moodle is extensible enough to add AI plugins and event exporters, but it is not architected as an AI-native data platform. PHP plugin development and deep Moodle conventions would slow Claude-assisted modernization relative to Python/React or Rails/React platforms.

Database and API. Moodle supports several RDBMS engines and has long-lived schema upgrade patterns, but its schema is LMS-first and historically broad. PostgreSQL is supported, but Supabase-native use would require an integration layer, not direct cohabitation.

Migration potential. Keep authentication integration, course containers, enrolment, assignments/quizzes, gradebook, files, calendar, and events. Rewrite frontend, ontology, graph, inference, recommendations, employer/career systems, analytics, and credentials outside Moodle. Estimated effort: medium-high.

Verdict. Excellent operational LMS skeleton, but long-term Syrka modernization would fight a large PHP monolith.

5.2 Canvas LMS

Overview. Canvas LMS is an open-source LMS developed by Instructure and used heavily in higher education. It is AGPL-3.0, primarily Ruby on Rails with JavaScript/React frontend components and PostgreSQL-oriented deployment. It has strong APIs, LTI support, mature course/assignment/gradebook operations, and corporate backing.

Architecture. Canvas is a large Rails monolith with substantial modular boundaries, background jobs, APIs, LTI support, and increasingly modern frontend islands.

Web / mobile / LTI / REST clients

v

Rails application: controllers, models, services, jobs

v

Domain modules: courses, enrollments, assignments, submissions, grades, discussions

v

PostgreSQL + Redis/cache + background workers + file services

Repository and implementation shape. The repository follows Rails conventions but is very large. Key areas include application models/controllers/views, frontend packages, database migrations, API controllers, background jobs, config, tests/specs, and deployment scripts. Canvas has a serious test culture but a high setup burden.

Core LMS coverage. Canvas is excellent for users, roles, courses, sections, enrolments, assignments, submissions, gradebook, rubrics, discussions, notifications, calendar, files, outcomes, analytics integrations, LTI, and permissions.

Extensibility and events. Canvas has excellent REST APIs, webhooks/live events in hosted contexts, and LTI. Its event and API surface is very useful for Syrka. However, self-hosted parity and operational complexity need due diligence. Deep modifications are harder because Instructure's SaaS roadmap shapes the code.

AI readiness. Stronger than Moodle for API-driven integration and modern frontend embedding. However, the platform remains a large Rails LMS. AI capability inference should be externalized.

Database and API. PostgreSQL alignment is a major advantage for Supabase-adjacent thinking, although Canvas's schema should not be merged into Syrka's Supabase schema. APIs are among the best in the group.

Migration potential. Keep courses, enrolments, assignments, submissions, gradebook, files, LTI, calendar, and API/event integration. Replace or bypass much of the frontend. Build graph/ontology/inference externally. Estimated effort: high because of Rails monolith complexity and AGPL implications.

Verdict. Technically attractive and modern enough, but less aligned than Open edX with modular learning content, event standardization, and independent learning runtime evolution.

Overview. Open edX is the open-source platform behind edX-style online learning. It is AGPL-3.0 and uses Python/Django, JavaScript, React micro-frontends, a modular monolith, independently deployable applications, XBlock extensibility, LTI, event tracking, analytics history, and a large provider/community ecosystem.

Architecture. Open edX is a modular monolith with service extraction patterns, micro-frontends, plugin hooks, XBlocks, event tracking, and a well-established deployment ecosystem through Tutor.

Learner / author / admin frontends

v

React MFEs + legacy web UI

v

Django LMS + Studio modular monolith

v

XBlocks, plugins, LTI tools, event tracking, APIs

v

MySQL + object storage + cache/search/queues

v

External analytics, data pipelines, IDAs, event consumers

Repository and implementation shape. openedx-platform contains LMS and Studio applications, Django apps, common libraries, XBlock runtime integration, courseware, grades, discussions integrations, content libraries, tests, configuration, management commands, APIs, and legacy plus modern frontend integration. The Open edX ecosystem also includes separate MFEs, events libraries, Tutor deployment, and IDAs.

Core LMS coverage. Open edX covers learners, instructors, course authoring, sequencing, content blocks, enrolment, assessments, grading, certificates, discussions integrations, files/assets, teams, cohorts, analytics/event tracking, and external tool integrations. It is less institutionally comprehensive than Moodle/Canvas in some traditional campus LMS workflows, but stronger as a scalable learning experience runtime.

Extensibility and events. This is the strongest candidate for Syrka's event-driven future. XBlocks permit new learning and assessment components. LTI permits external tools. The tracking/event tradition and newer event-bus direction are well aligned with capability inference. Educational events can be normalized and streamed into Syrka's canonical evidence ledger.

AI readiness. Open edX is the best fit for AI-native augmentation because Python/Django integrates naturally with LLM orchestration, embeddings, graph/ontology services, evaluation pipelines, and ML tooling. React MFEs are more compatible with Syrka's modern UX trajectory than older PHP/Java template stacks.

Database and API. The default database is MySQL, which is a mismatch with Supabase PostgreSQL. This should be addressed by not merging schemas. Syrka should use Open edX as an operational LMS bounded context and stream canonical events into Supabase/PostgreSQL and graph stores.

Migration potential. Keep course runtime semantics, XBlock/LTI extension points, assessment primitives, event tracking concepts, certificates where useful, content libraries, enrolment

concepts, and selected APIs. Rewrite identity/profile unification, ontology, graph, inference, career intent, market signals, government/employer dashboards, job passport, verified credential ledger, and public profiles as Syrka-native services. Estimated effort: high initially, but best long-term compounding value.

Verdict. Best architectural foundation if treated as a bounded operational substrate rather than a permanent monolithic core.

5.4 Chamilo

Overview. Chamilo is a PHP/MySQL open-source LMS focused on ease of use and accessibility. It is GPL-3.0+ and has been used globally since 2010, with a smaller but active community.

Architecture. Chamilo is a simpler PHP LMS, moving in a more Symfony/Twig direction in newer versions. It is less complex than Moodle/Canvas/Open edX and easier to self-host, but the ecosystem and architectural primitives are thinner.

Core LMS coverage. Chamilo covers users, courses, enrolments, learning paths, tests, documents, assignments, reporting, skills-related features, and basic communication. It is practical for conventional e-learning.

Extensibility and events. Extensibility exists, but it is not as broad or event-rich as Moodle/Open edX/Canvas for a capability inference system. Syrka would quickly outgrow it.

AI readiness. Low-to-medium. Simplicity helps initial modification but hurts long-term platform ambition.

Migration potential. Keep little beyond basic course/user/content concepts. Most Syrka modules would be new systems. Estimated effort: medium initially, very high over time.

Verdict. Too small and conventional for Syrka's intended infrastructure role.

5.5 ILIAS

Overview. ILIAS is a mature GPL-3.0 PHP LMS with strong European institutional adoption, governance, assessment, repository, and permission capabilities.

Architecture. ILIAS is a mature modular monolith centered around a repository-object model, services, components, plugins, and deep role/permission concepts.

Core LMS coverage. Strong in users, roles, repository objects, courses/groups, learning modules, tests, surveys, exercises, SCORM, files, communication, calendar, reporting, and permissions.

Extensibility and events. ILIAS has plugin slots and a disciplined component architecture, but its conventions are specialized. Event-driven capability inference would be possible but not natural.

AI readiness. Medium. Strong institutional data model, weaker modern AI/React/Python ecosystem fit.

Migration potential. Keep repository, role, assessment, file, and course structures if adopting it. Rewrite most Syrka intelligence services. Estimated effort: high.

Verdict. Mature and serious, but less globally extensible and less AI-native than Open edX.

5.6 Sakai

Overview. Sakai is a Java/Tomcat open-source LMS and collaboration environment governed through the Apereo ecosystem. It is licensed under Educational Community License 2.0, which is more permissive than GPL/AGPL alternatives.

Architecture. Sakai is a service-oriented Java application with tools depending on shared services for site, user, authorization, content, events, assignments, gradebook, and collaboration.

Portal / tools

v

Sakai services and kernel

v

Tool implementations: assignments, gradebook, tests, forums, resources, calendar

v

RDBMS + content/file storage + Tomcat runtime

Core LMS coverage. Strong in higher-ed collaboration, sites/courses, assignments, gradebook, tests/quizzes, resources, announcements, calendar, messaging, and permissions.

Extensibility and events. The service/tool model is extensible and the license is attractive. However, the modern frontend and AI ecosystem fit is weaker than Open edX. Java services can be robust but slower for rapid AI-native iteration in Syrka's current stack.

AI readiness. Medium. Good for enterprise Java integration; less ideal for LLM/embedding/graph experimentation and modern UX replacement.

Migration potential. Keep assignments, gradebook, site/user/role services, resources, and event concepts. Rewrite Syrka intelligence services and UX. Estimated effort: high.

Verdict. Solid and permissively licensed, but lower momentum for the desired product architecture.

6. Detailed scorecard

Scores are 1-10, where 10 is strongest alignment with Syrka's long-term architecture.

Criterion	Moodle	Canvas	Open edX	Chamilo	ILIAS	Sakai
-----------	--------	--------	----------	---------	-------	-------

	---	---	---	---	---	---
--	-----	-----	-----	-----	-----	-----

Architecture	7	8	9	5	7	7
--------------	---	---	---	---	---	---

Code quality	7	8	7	5	7	7
--------------	---	---	---	---	---	---

Documentation	9	7	8	6	7	7
---------------	---	---	---	---	---	---

Developer experience	7	6	7	6	5	6
----------------------	---	---	---	---	---	---

Scalability	8	9	9	5	7	8
-------------	---	---	---	---	---	---

Security	8	8	8	6	8	8
----------	---	---	---	---	---	---

Testing	8	8	8	6	7	7
---------	---	---	---	---	---	---

Performance	7	8	8	6	7	7
-------------	---	---	---	---	---	---

API quality	7	9	8	5	6	6
-------------	---	---	---	---	---	---

Plugin architecture	10	7	9	6	8	7
---------------------	----	---	---	---	---	---

Community	10	8	8	6	6	6
Maintainability	7	7	7	6	6	7
AI readiness	6	7	9	4	5	6
Knowledge graph compatibility	6	7	9	4	5	6
Ontology integration	7	7	9	5	6	6
Ease of forking	6	5	5	8	6	7
Ease of modernization	5	7	8	6	5	5
Suitability for Vercel	3	4	6	3	3	3
Suitability for Supabase PostgreSQL	6	8	5	4	4	6
Suitability for Claude-assisted development	6	7	8	6	5	6
Long-term technical debt risk	6	6	7	4	5	6
Weighted Syrka fit	7.2	7.7	8.4	5.6	6.2	6.6

7. Syrka module compatibility

| Syrka module | Moodle | Canvas | Open edX | Chamilo | ILIAS | Sakai | Reuse vs. build |
 |---|---|---|---|---|---|---|
 | Syrka LMS | Strong | Strong | Strong | Adequate | Strong | Strong | Reuse LMS primitives |
 | Syrka Ontology | External | External | Best external fit | External | External | External | Build new
 ontology service |
 | Human Capability Graph | External | External | Best event feed | External | External | External | Build
 graph service |
 | Capability Inference Engine | External | External | Best Python/event fit | External | External |
 External | Build new inference service |
 | Career Intent Engine | External | External | External | External | External | External | Build new
 service |
 | Syrka Odyssey | External UX | External UX | Strong learning-path basis | Weak | Medium | Medium | Build
 Syrka-native UX, reuse course evidence |
 | Market Signal Engine | None | None | None | None | None | None | Build new service |
 | National Vision Layer | None | None | None | None | None | None | Build new service |
 | Research Intelligence | None | None | Possible event/content input | None | None | Collaboration input
 | Build new service |
 | GitHub Intelligence | External tool | External tool | LTI/XBlock fit | External | External | External |
 Build new integration |
 | LinkedIn Intelligence | External | External | External | External | External | External | Build new
 integration |
 | Capability Dashboard | Report plugin | API dashboard | Event dashboard | Basic | Report plugin | Tool |
 Build Syrka-native dashboard |
 | Portfolio Generator | Limited | Outcomes/files | Certificates/content evidence | Limited | Repository
 objects | Resources | Build new generator |
 | Opportunity Engine | None | None | None | None | None | None | Build new service |
 | Research Collaboration Engine | Forum/wiki input | Discussion input | Teams/discussions input | Basic |
 Collaboration input | Strong collaboration input | Build new service |
 | Employer Dashboard | None | Limited admin patterns | None | None | None | None | Build new dashboard |
 | Faculty Dashboard | Existing reports | Strong | Strong course analytics | Basic | Strong | Strong |
 Reuse data, build Syrka UX |
 | Curriculum Intelligence | Course metadata | Course/outcomes | Strong content model | Medium | Strong
 repository | Medium | Build new AI layer |
 | Institutional Analytics | Strong | Strong | Strong event base | Basic | Strong | Medium | Reuse data
 feeds, build analytics |
 | Job Passport | None | Credentials input | Certificate/evidence input | None | Certificates input | None
 | Build new passport/ledger |
 | Verified Credential Ledger | Badges/certs | Credentials input | Certificates input | Limited |
 Certificates | Limited | Build new ledger |

| Workforce Mobility Engine | None | None | None | None | None | None | Build new service |
 | Syrka Corridors | None | None | None | None | None | None | Build new service |
 | Public Capability Profile | Profile extension | Profile/API | Profile/evidence feed | Profile | Profile
 | Profile | Build new public profile |
 | AI-Native Learning Analytics | Logs | Events/APIs | Best fit | Weak | Medium | Medium | Build AI
 analytics on canonical events |

8. Event architecture conclusion

For Syrka, event architecture is the decisive criterion. The selected foundation must emit granular, semantically useful educational events and permit clean interception without modifying every activity implementation.

Required event families:

- Identity and enrolment events.
- Course, unit, module, and learning-path progress events.
- Assignment creation, submission, resubmission, grading, rubric, and feedback events.
- Quiz/problem attempt events including timing, correctness, hints, retries, and mastery.
- Discussion, peer review, team, collaboration, and research events.
- File, portfolio, repository, external tool, and project artifact events.
- Attendance and synchronous participation events.
- Credential, certificate, badge, and verification events.

Ranking for Syrka event architecture:

1. Open edX - strongest because of event tracking heritage, XBlock runtime, LTI compatibility, and Python ecosystem.
2. Canvas - excellent APIs and event possibilities, but hosted/self-hosted differences and Rails monolith concerns.
3. Moodle - mature event API and plugins, but less AI-native architecture.
4. Sakai - service/tool events are useful, but modernization cost is higher.
5. ILIAS - strong permissions/repository model, less natural event-stream architecture.
6. Chamilo - simplest, but least capable for deep capability inference.

9. Target architecture after adopting Open edX hybrid

Syrka Next.js applications
 student / faculty / employer
 ministry / public profiles

?
 Syrka API gateway / BFF
 auth, policy, aggregation

??
 Open edX bounded Syrka-native
 LMS context intelligence services
 LMS + Studio ontology, graph, AI
 XBlock + LTI career, market, jobs

??

Canonical Syrka Event Bus
evidence + capability events

? ?

Supabase/Postgres Graph/vector stores
operational views capability graph

Open edX should be isolated behind an anti-corruption layer. Syrka's canonical events should not mirror Open edX tables one-to-one. They should express Syrka concepts: learner, evidence item, capability, context, assessor, confidence, provenance, validity window, market alignment, and credential claim.

10. Components to reuse

From Open edX:

- Course authoring and course runtime concepts.
- XBlock component model for new learning objects, assessments, simulations, AI exercises, and project evidence capture.
- LTI integration patterns for external tools.
- Learner progress and grading primitives.
- Content sequencing and courseware concepts.
- Certificates only as an input to Syrka's future credential ledger, not as the final credential architecture.
- Event tracking concepts and event consumers.
- Existing deployment practices as an operational reference, preferably through Tutor during evaluation/prototyping.

From Canvas/Moodle as reference patterns only:

- Canvas API design, assignment/submission ergonomics, gradebook UX patterns, LTI maturity.
- Moodle plugin taxonomy and role/capability lessons.

Do not copy multiple codebases into Syrka. Use them as architecture references unless a specific license-compatible module is intentionally isolated and reviewed.

11. Components to rewrite or build as Syrka-native systems

The following should be built outside the LMS foundation:

- Syrka Ontology.
- Human Capability Graph.
- Capability Inference Engine.
- Career Intent Engine.
- Syrka Odyssey user experience.
- Market Signal Engine.
- National Vision Layer.
- Research Intelligence

- GitHub Intelligence.
- LinkedIn Intelligence.
- Employer Dashboard.
- Faculty Dashboard beyond conventional LMS reports.
- Curriculum Intelligence.
- Institutional Analytics.
- Job Passport.
- Verified Credential Ledger.
- Workforce Mobility Engine.
- Cross-Border Capability Corridors.
- Public Capability Profile.
- AI-native analytics, recommendation, ranking, fairness, and explainability services.

12. Why alternatives were rejected

Build completely from scratch

Rejected because Syrka would spend years rebuilding commodity LMS operations: enrolments, assignments, quizzes, gradebooks, content handling, course authoring, roles, permissions, and LTI. The strategic value is not a blank LMS; it is capability inference and workforce mobility. Scratch-building the operational LMS delays the actual company-defining platform.

Fork Moodle

Rejected as primary direction because Moodle's strengths are plugin breadth and institutional maturity, not AI-native architecture. It is viable as an operational LMS, but a long-lived Syrka fork would inherit a large PHP monolith and modernization drag.

Fork Canvas LMS

Rejected as primary direction despite strong APIs and PostgreSQL alignment because it is a large Rails product shaped by Instructure's SaaS priorities, has AGPL obligations, and is less naturally aligned with modular learning component authoring and Python AI pipelines than Open edX.

Fork Open edX directly

Rejected as stated because a permanent direct fork risks trapping Syrka inside a heavy monolith. The accepted decision is Open edX hybrid: adopt, wrap, instrument, and extract.

Fork Chamilo

Rejected because it is too lightweight for Syrka's long-term capability infrastructure and event/AI ambitions.

Fork ILIAS

Rejected because it is mature but specialized, PHP-heavy, and less compatible with the modern AI-native, graph-driven, Vercel/Next.js/Supabase-oriented Syrka direction.

Fork Sakai

Rejected because its license and service orientation are attractive, but the Java/Tomcat architecture and older frontend model are not the best fit for rapid AI-native product

evolution.

13. Phased migration strategy

Phase 0 - Due diligence hardening, 2-4 weeks

- Resolve network access and perform full local clones of all six repositories.
- Run repository-level static analysis, test sampling, dependency audits, and local bootstraps for Open edX and Canvas.
- Validate AGPL obligations with counsel.
- Produce a canonical Syrka learning event schema.
- Identify the minimum viable Open edX installation profile.

Phase 1 - Bounded Open edX pilot, 6-10 weeks

- Deploy Open edX in an isolated environment.
- Integrate Syrka identity through SSO/OIDC where feasible.
- Create an event export pipeline into Syrka's Supabase/PostgreSQL environment.
- Build initial XBlock or LTI proof of concept for capability evidence capture.
- Map Open edX events to Syrka evidence entities.
- Keep learner-facing Syrka UX in Next.js where possible.

Phase 2 - Canonical capability event ledger, 8-12 weeks

- Define canonical events: learning.activity.started, assessment.submitted, assessment.evaluated, capability.evidence.observed, credential.claim.issued, collaboration.signal.detected, and related schemas.
- Implement event ingestion, validation, provenance, idempotency, and replay.
- Store operational projections in Supabase/PostgreSQL.
- Store capability relationships in a graph database or graph-compatible layer.
- Add vector indexes for evidence retrieval and LLM grounding.

Phase 3 - Syrka-native intelligence services, 12-20 weeks

- Build the Syrka Ontology service.
- Build Human Capability Graph service.
- Build Capability Inference Engine with transparent confidence/provenance.
- Build Career Intent Engine and Odyssey recommendations.
- Integrate market signals, research intelligence, GitHub evidence, and employer feedback.

Phase 4 - UX replacement and module extraction, 20-40 weeks

- Replace learner, faculty, employer, and ministry workflows with Syrka-native Next.js experiences.
- Use Open edX primarily as runtime/content/assessment infrastructure where still valuable.
- Extract or replace Open edX modules that conflict with Syrka's product model.
- Introduce Syrka Job Passport and Verified Credential Ledger.

Phase 5 - Human Capital Infrastructure, ongoing

- Operate cross-border capability corridors.
- Provide verified public profiles.
- Offer workforce mobility analytics to governments and employers.
- Maintain LMS compatibility through LTI event adapters and standard credential formats.

14. Consequences

Positive

- Syrka avoids wasting years rebuilding commodity LMS functionality.
- Open edX provides the best combination of scalable learning runtime, extensibility, events, Python ecosystem, and modern frontend direction.
- The architecture protects Syrka's strategic systems from LMS lock-in.
- XBlocks and LTI provide practical hooks for AI-native assessment and evidence capture.
- The anti-corruption layer gives Syrka freedom to modernize incrementally.

Negative

- Open edX is operationally heavy.
- The default MySQL data layer does not align with Supabase PostgreSQL.
- AGPL obligations require careful legal/product planning.
- The team must maintain a boundary between Open edX domain concepts and Syrka canonical capability concepts.
- Full local clone/test due diligence must still be completed when network access permits.

15. Final recommendation

Choose the hybrid approach: use Open edX as the primary LMS foundation and learning runtime, but build Syrka's intelligence, ontology, graph, career, credential, employer, government, and workforce mobility systems as independent Syrka-native services.

Open edX is selected because it has the strongest alignment with Syrka's need for modular learning experiences, event-rich educational telemetry, AI/ML-friendly Python infrastructure, React-compatible modernization, and scalable online learning patterns. Moodle and Canvas are stronger conventional LMS products in certain institutional workflows, but Syrka's defining advantage is not conventional LMS completeness. Syrka's advantage is converting educational activity into trustworthy capability intelligence and workforce mobility infrastructure.

The company should not bet its future on a permanent fork of any LMS monolith. It should use Open edX to accelerate the operational layer, then systematically move the center of gravity to Syrka's canonical event ledger, ontology, graph, inference, passport, and corridor services.